Jane Street Market Prediction

Link - https://www.kaggle.com/c/jane-street-market-prediction

Prathyusha Sathineni

Introduction

Jane street is a trading firm, and their philosophy is to maximize profits.

Jane Street market prediction is a Kaggle competition. In this trading challenge, we must predict the action value for a trading opportunity.

Action value: 1 to make the trade and 0 to pass on it

About Dataset

- It’s a 6 GB dataset.
- Each row in the dataset represents a trading opportunity for which we will be predicting an action value.
- The following is the list of files we will be using for our analysis
    - train.csv
    - example_test.csv
    - features.csv

About each file

train.csv

- Contains historical data and returns
- 2390491 rows × 138 columns
- Attributes:
  - feature_{0…129} - anonymized features.
  - resp, resp_{1,2,3,4} - represents returns over different time horizons.
  - weight - represents weight associated with each trade
  - date - represents the day of trade
  - ts_id - represents time ordering

example_test.csv

- A mock test set that represents the structure of the unseen test set. We will not be using this test dataset directly to test the accuracy .
- 15219 rows × 133 columns only.
- It has all the attributes same as that of train.csv, except resp values

features.csv

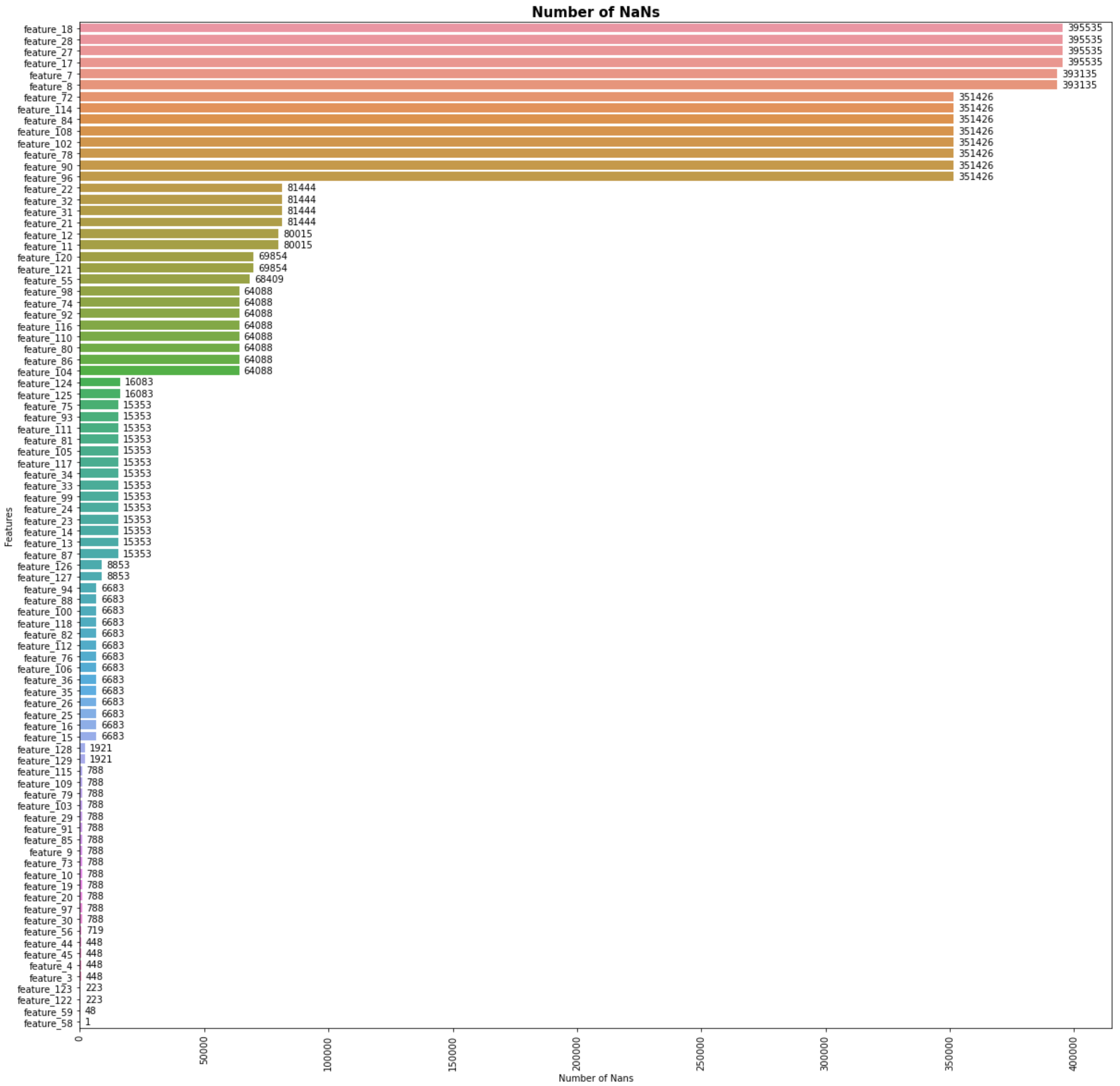
- Metadata about the anonymized features.
- 130 rows × 29 columns
- It provides information about anonymized tags {0,….,28} - Boolean values, for each feature

Data Preprocessing

- Step 1: Handling Null Values
- Step 2: Handling weight records
- Steps 3: Data reduction based on trade returns

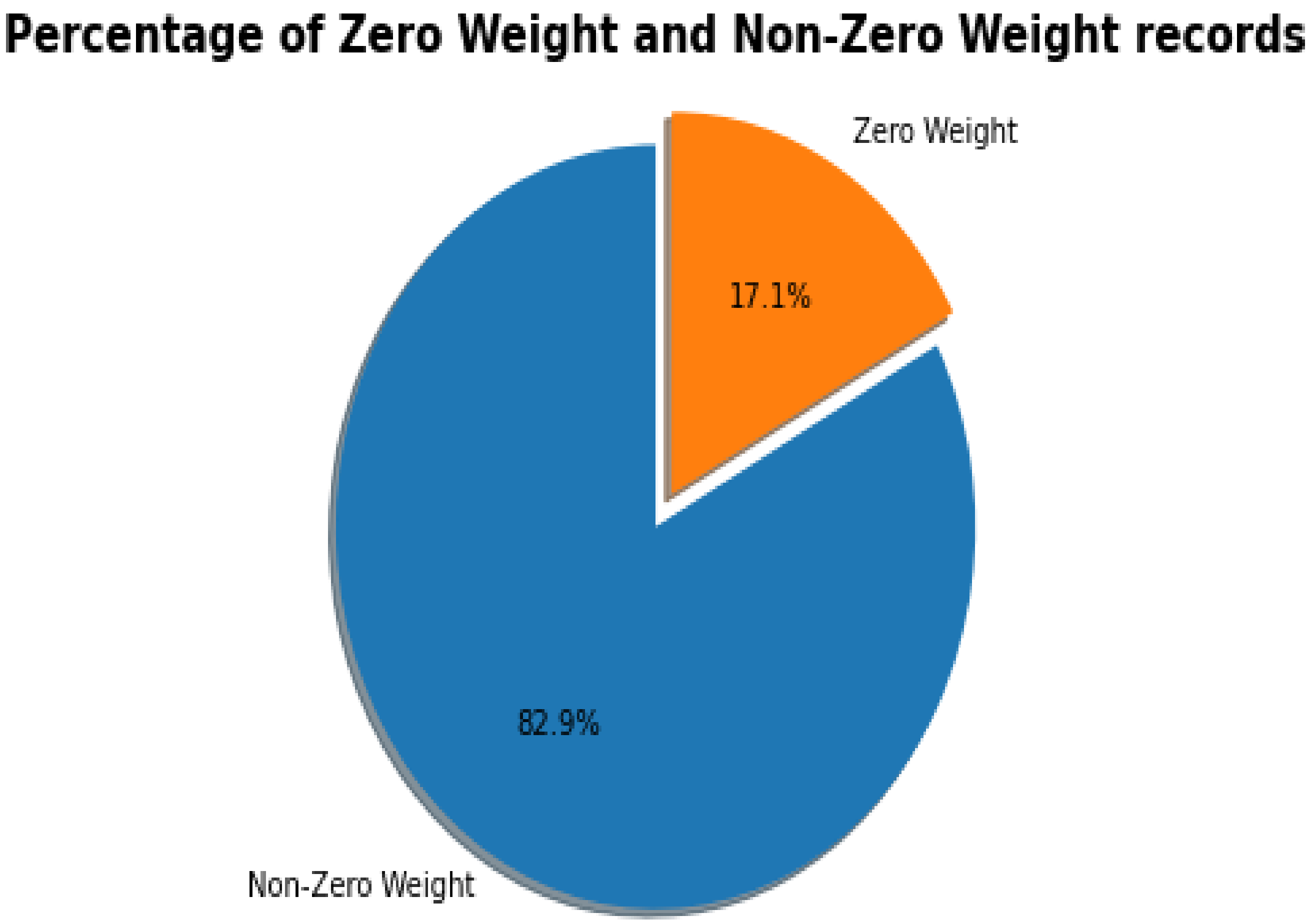
Step 1: Handling Null values

- Maximum : 395535 null values
- Minimum : 1 null value
- Removing null values leads to overfitting
- So, imputed null values with mean



Step 2: Handling weight records

- Weight zero records indicate gain from that trade is 0.
- Therefore, we can drop records with zero weight.



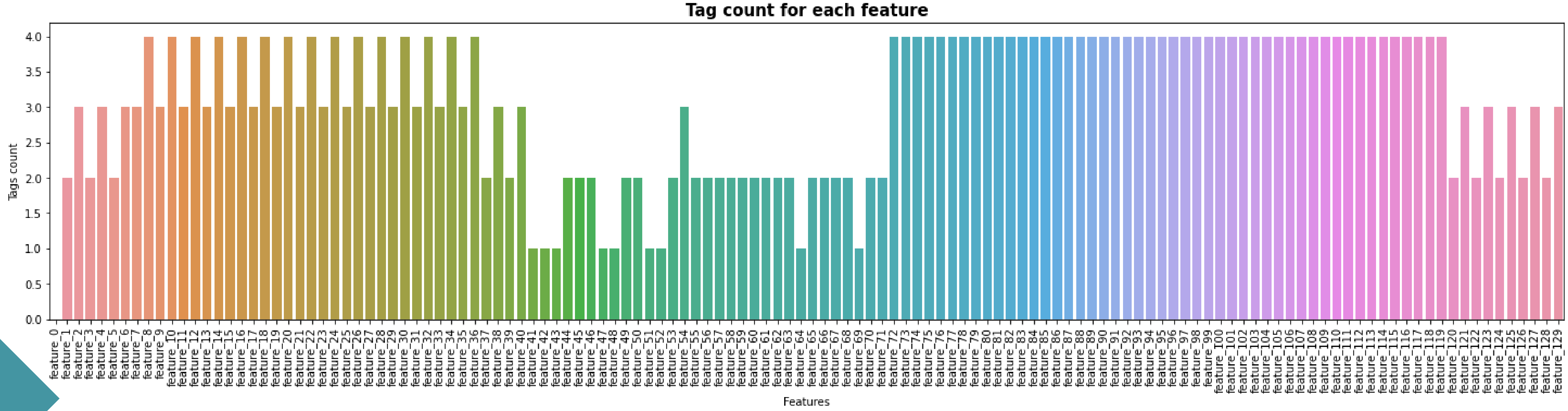
Step 3: Data Reduction based on trade returns

- In dataset we have transactions for 500 days.
- Weight and resp together represents a return.
- Observed that trade returns started increasing drastically after 85 days.
- Therefore, we can exclude records before 85th day from the dataset.

Feature Selection

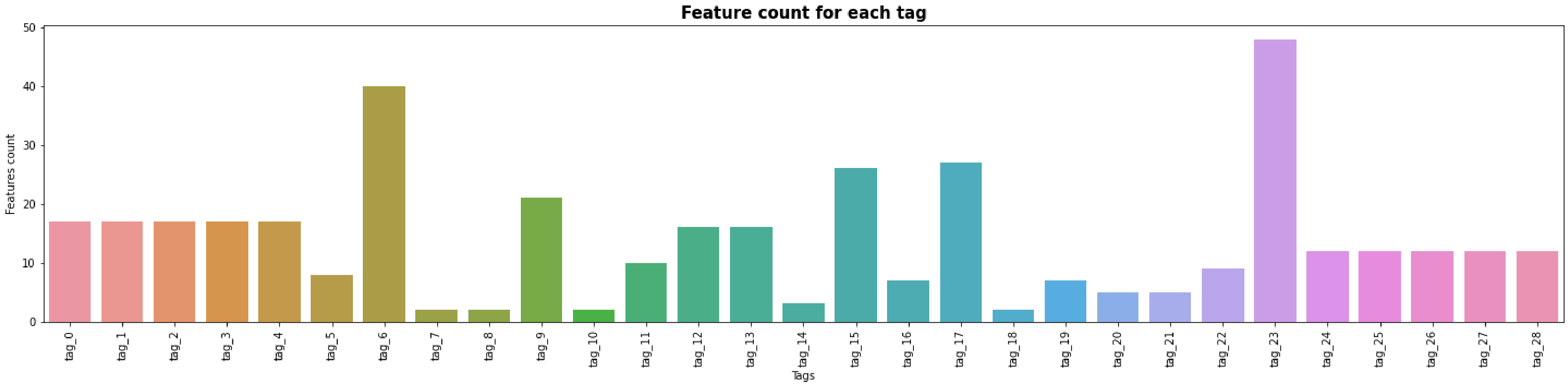
- Process 1 : Exploring features.csv
- Process 2: Correlation Heatmap
- Process 3: Checking Outliers
- Process 4: Random Forest Feature Importance Function

Process 1: Exploring features.csv



- The feature_0 has 0 tags.
- 63 features - feature_{8,10,12,14,16,18,20,22,24,26,28,30,32,34,36,72,.....,119} has 4 tags.
- 26 features - feature_{2,4,6,7,9,11,13,15,17,19,21,23,25,27,29,31,33,35,38,40,54,121,123,125,127,129} has 3 tags.
- 31 features - feature_{1,3,5,37,39,44,45,46,49,50,53,55,..,63,65,66,67,68,70,71,120,122,124,126,128} has 2 tags.
- Remaining 9 features, feature_{41,42,43,47,48,51,52,64,69,} has only 1 tag.

Process 1: Cont.



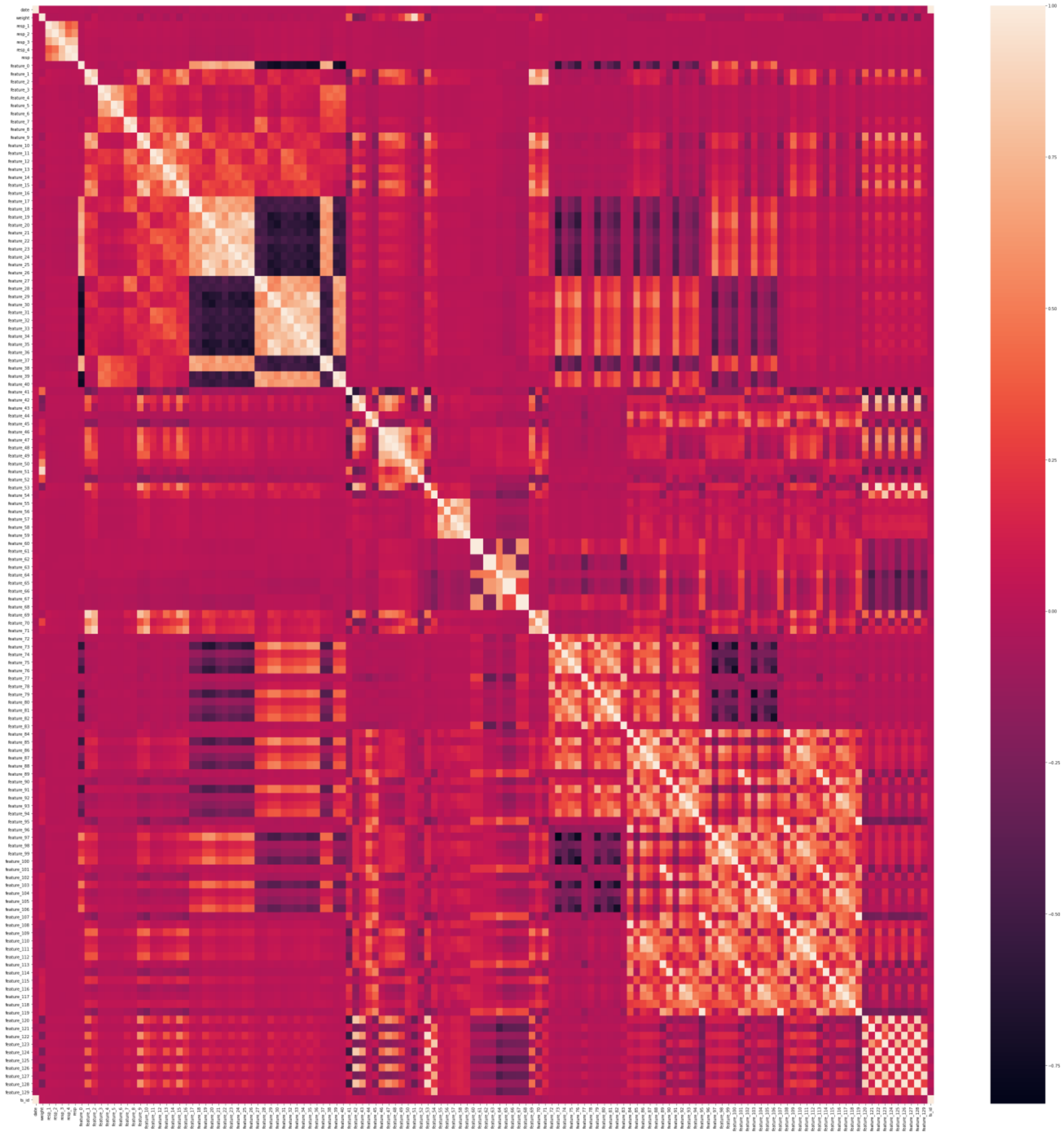
- Approximately 40+ features have tag_6 and tag_23.
- 20+ features have tag_9, tag_15 and tag_17.
- The First 5 tags and last 5 tags have the same number of features. 17, and 12 respectively.

Based on these observations, it is hard to select important features.

Process 2: Correlation Heatmap

- There are a lot of multicollinearities

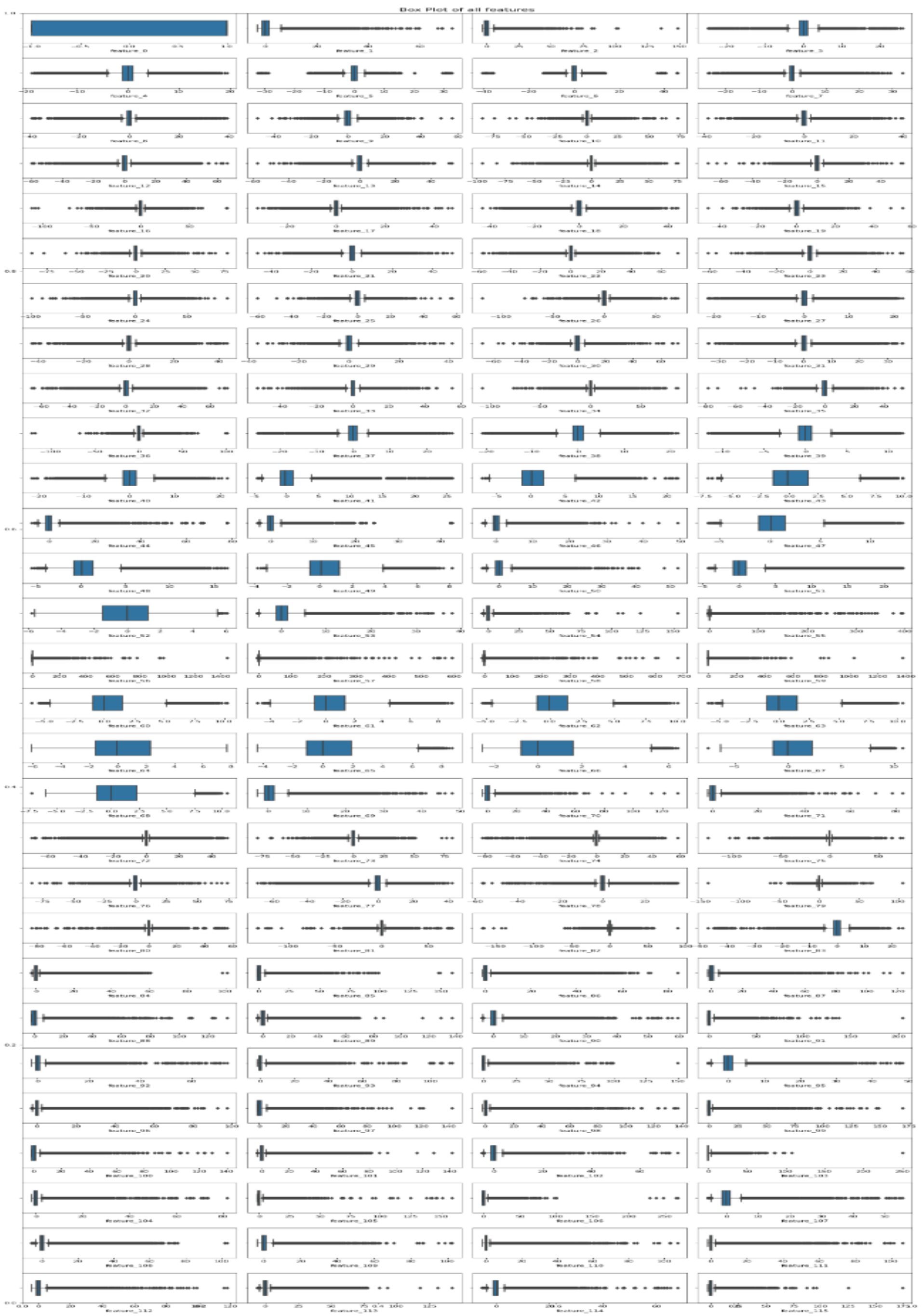
It's hard to get the important features from these clusters.



Process 3: Checking Outliers

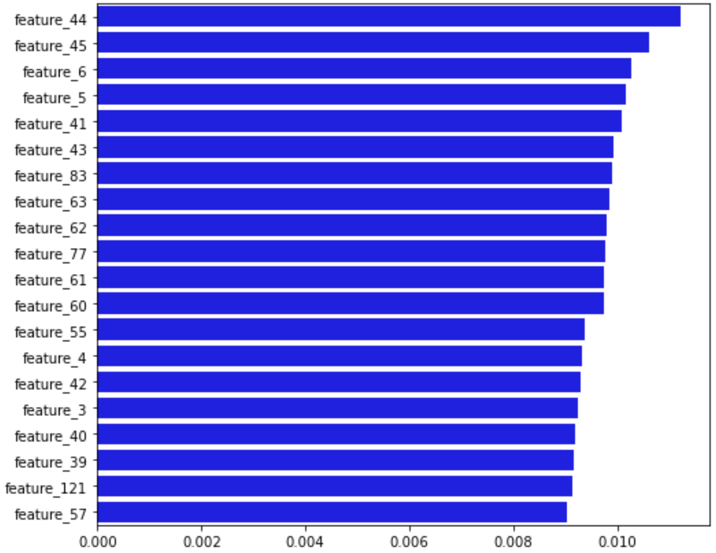
However, we can observe that all the variables have many outliers with larger values, and without domain knowledge, it is inappropriate to eliminate attributes based on outliers.

Therefore, It's hard to get the important features from outliers

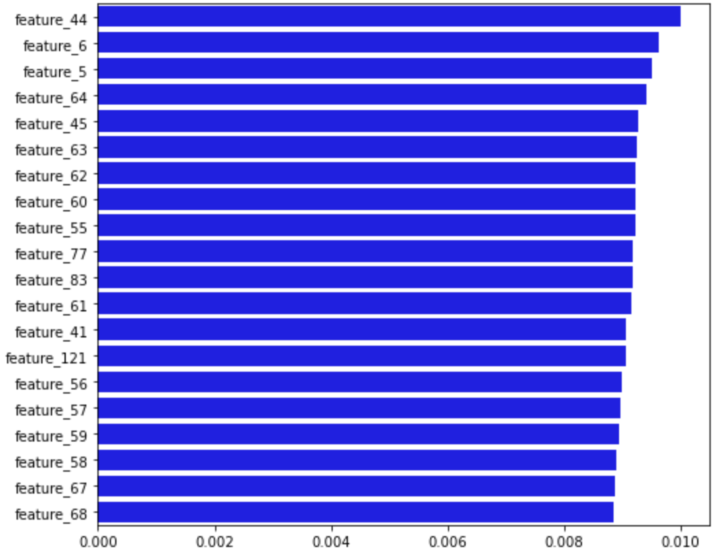


Process 4: Random Forest Feature Importance function

Action Variable Approach 1 : If **resp >0** then action = 1, else action =0



Action Variable Approach 2: : If **(resp * weight) & (resp_1 * weight) & (resp_2 * weight) & (resp_3 * weight) & (resp_4 * weight) > 0** then action = 1 else action = 0.

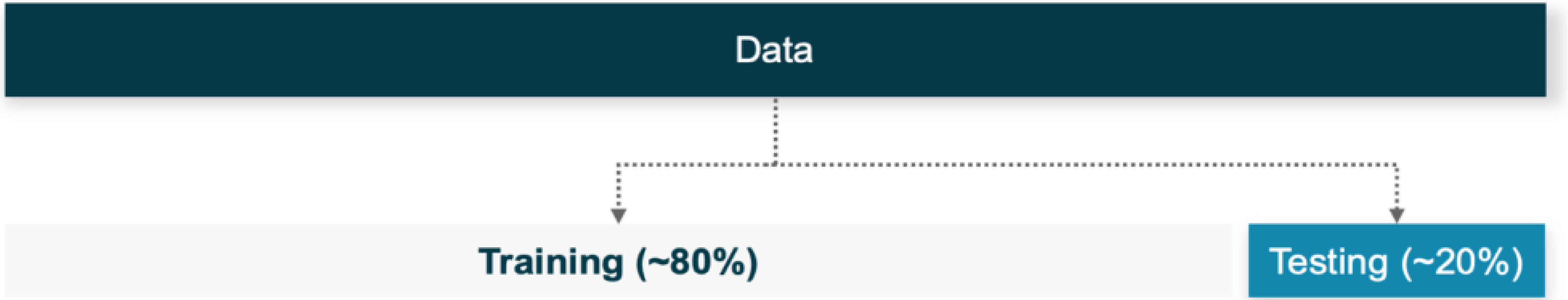


Interesting observations between important features of 2 approaches:

- feature_44 is the first important variable.
- feature_6 ranks in the top 3 in both.
- There are 14 same features.

Hence, it is apt to select the features using the random forest feature importance method

Data Partitioning (train.csv)



Training and Testing set contains only important features and the target variable

Model Evaluation Metrics

- Depends on implementation plan.
- Will be using classification models for prediction, we will be using the **confusion matrix** as metrics for model evaluation.
- Confusion matrix report provides us with the dimensions like
  - Accuracy
  - Precision
  - Recall
  - F1-score
- In this project, we will be mainly using the accuracy dimension for the model evaluation.
- Accuracy gives the proportion of the total number of predictions that were correct.

Models

	Approach 1	Approach 2
Action value	If resp >0 then action = 1, else action =0	If (resp * weight) & (resp_1 * weight) & (resp_2 * weight) & (resp_3 * weight) & (resp_4 * weight) > 0 then action = 1 else action = 0.
Multiple Layer Perceptron	Confusion matrix: $\begin{bmatrix} 10019 & 615424 \\ 10206 & 621483 \end{bmatrix}$ Accuracy: 50.23%	Confusion matrix: $\begin{bmatrix} 924363 & 0 \\ 332769 & 0 \end{bmatrix}$ Accuracy: 73.53%
Decision Tree	Confusion matrix: $\begin{bmatrix} 374822 & 250621 \\ 252364 & 379325 \end{bmatrix}$ Accuracy: 59.98%	Confusion matrix: $\begin{bmatrix} 677816 & 246547 \\ 225887 & 106882 \end{bmatrix}$ Accuracy: 62.42%
Random Forest Tree	Confusion matrix: $\begin{bmatrix} 334236 & 291207 \\ 310638 & 321051 \end{bmatrix}$ Accuracy: 52.13%	Confusion matrix: $\begin{bmatrix} 924363 & 0 \\ 332769 & 0 \end{bmatrix}$ Accuracy: 73.53%

Model Selection

- The accuracy of all the models is high in approach 2 when compared to approach 1.
- **Interesting observation** : In approach 2, both the model's multiple layer perceptron and random forest tree have the same accuracy.
- These 2 models stand out to be the highest accuracy models among all six models with 73.53% accuracy.
- Random Forests often have little performance gain when a certain amount of data is reached, while Neural Networks usually benefit from large amounts of data and continuously improve the accuracy.
- Hence, from the above discussion, we can consider the 2nd approach as the best approach.
- **Multiple layer perceptron to be the most accurate model** that can be used for predicting the action value.

Conclusion

- We can conclude that approach 2 gives better accuracy for all three models.
- Among these three models, multiple layer perceptron would be the best model to predict the action value.
- The hardest part of this project was handling the 5 GB data.
    - To run various models on 5 GB data, we need more memory, and the system crashed several times.
    - To overcome this problem, in the future, we can use AWS and try various models.
- One more hardest part was the feature selection process.
    - Since this is competition data, all the features were anonymized.
    - Feature selection process can be improved by using any of the 3 approaches listed below:
        - By doing more analysis on the clusters formed in the correlation heatmap.
        - By seeing the feature count plot, we can understand that each tag has a set of features. Possibly we can do cluster analysis by creating a cluster for each tag. Therefore we have to analyze 28 groups.
        - By doing more research and identifying the relationship between feature and tags. So that by using the tags count of each feature possibly we can extract important features.

Thank You